

CLIENTE - SERVIDOR

El servidor es el programa central que gestiona todo. Cuando alguien se conecta, le pide que elija un nombre de usuario. Si ese nombre ya está en uso, el servidor le asigna uno similar automáticamente. Una vez conectado, el usuario puede usar varios comandos: ver la fecha y hora, ver quién está conectado, enviar mensajes privados a alguien en particular, enviar mensajes a todos al mismo tiempo, resolver operaciones matemáticas, y jugar un juego de preguntas y respuestas de opción múltiple (A, B, C o D) con 10 preguntas sobre cultura general. Cuando alguien se desconecta, el servidor avisa al resto.

El cliente es el programa que usa cada persona para conectarse al servidor. Tiene dos tareas al mismo tiempo: una parte se encarga de leer lo que escribe el usuario y enviarlo al servidor, y otra parte espera y muestra los mensajes que llegan desde el servidor. Si el usuario escribe "SALIR", la conexión se cierra. En resumen, es un chat multi-usuario por consola donde varias personas pueden conectarse, chatear, mandarse mensajes privados y jugar juntos, todo coordinado por un servidor central.

Método del cliente: **main** -> Se conecta al servidor, lanza un hilo secundario que escucha mensajes entrantes, y en el hilo principal envía lo que escribe el usuario.

```
//Joaquín Devige.  
//Legajo: 114638.  
  
package hilosysockets;  
  
import java.io.*;  
import java.net.*;  
import java.util.Scanner;  
  
public class HilosYSockets {  
  
    public static void main(String[] args) throws Exception {  
  
        Scanner teclado = new Scanner(System.in);  
        Socket sc = new Socket("127.0.0.1", 5500);  
        System.out.println("¡Conectado al servidor!\n");  
  
        //Estas clases nos permiten usar métodos como readUTF() y writeUTF()  
        //para enviar y recibir texto de forma directa.  
  
        DataInputStream entrada = new DataInputStream(sc.getInputStream());  
        DataOutputStream salida = new DataOutputStream(sc.getOutputStream());
```

```
//Hilo lector.  
//Se ejecuta en paralelo al hilo principal.  
//Su única tarea: esperar mensajes del servidor e imprimirlos.  
Thread lector = new Thread() -> {  
    try {  
        while(true){  
            //Se queda esperando mensajes del servidor y los imprime en pantalla.  
            //El metodo readUTF() detiene este hilo especifico hasta que llega un mensaje,  
            //lo imprime y vuelve a esperar.  
            String msg = entrada.readUTF();  
  
            System.out.println("\n[SERVIDOR] " + msg);  
        }  
    } catch (Exception e) {  
        System.out.println("\nConexión cerrada por el servidor.");  
    }  
};  
  
//Un hilo Daemon (demonio) es un hilo "secundario".  
//Si el hilo principal termina, el setDaemon() mata a todos los hilos Daemon automáticamente y cierra el programa.  
lector.setDaemon(true); //Se cierra automáticamente al terminar el programa.  
lector.start();
```

Joaquín Devige CLIENTE - SERVIDOR

```
//Hilo principal (escritura).
//Lee lo que escribe el usuario y lo envía al servidor.
while(true){

    String respuesta = teclado.nextLine().trim();

    //Si el usuario apretó enter sin escribir nada,
    //salta a la siguiente iteración del bucle.
    if(respuesta.isEmpty()){
        continue; //Ignorar líneas vacías.
    }

    salida.writeUTF(respuesta); //Le manda la respuesta del usuario al servidor.

    if(respuesta.equalsIgnoreCase("SALIR")){
        break;
    }

}

sc.close();
System.out.println("Conexión cerrada.");
}
```

Métodos del servidor: PREGUNTAS, ConcurrentHashMap y main. Arranca el servidor en el puerto 5500 y crea un hilo nuevo por cada cliente que se conecta, Se crea un concurrentHashMap para almacenar los clientes y un array con las preguntas..

```
//Joaquín Devige.
//Legajo: 114638.

package hilosysockets;

import java.io.*;
import java.net.*;
import java.text.SimpleDateFormat;
import java.util.*;
import java.util.concurrent.*;

public class HilosYSocketsServer {

    //Mapa compartido: nombre: salida de cada cliente.
    public static ConcurrentHashMap<String, DataOutputStream> clientes = new ConcurrentHashMap<>();

    //Banco de preguntas.
    public static final String[][] PREGUNTAS = {
        {"¿Cuánto es 5 + 3?", "A)6", "B)8", "C)9", "D)7", "B"},
        {"¿Capital de Francia?", "A)Roma", "B)Madrid", "C)París", "D)Berlín", "C"},
        {"¿Lados de un hexágono?", "A)5", "B)7", "C)8", "D)6", "D"},
        {"¿Quién escribió el Quijote?", "A)Lope", "B)Cervantes", "C)Quevedo", "D)Góngora", "B"},
        {"¿Planeta más grande?", "A)Saturno", "B)Neptuno", "C)Júpiter", "D)Urano", "C"},
        {"¿Cuánto es 12 x 12?", "A)144", "B)124", "C)132", "D)148", "A"},
        {"¿Año llegada del hombre a la Luna?", "A)1965", "B)1972", "C)1969", "D)1971", "C"},
        {"¿Océano más grande?", "A)Atlántico", "B)Índico", "C)Ártico", "D)Pacífico", "D"},
        {"¿Colores del arcoíris?", "A)5", "B)6", "C)7", "D)8", "C"},
        {"¿Símbolo químico del oro?", "A)Go", "B)Or", "C)Ag", "D)Au", "D"}
    };
}
```

```
public static void main(String[] args) throws Exception {

    ServerSocket servidor = new ServerSocket(5500);
    System.out.println("Servidor iniciado en puerto 5500...");

    while (true){

        Socket cliente = servidor.accept();
        System.out.println("Nueva conexión desde:" + cliente.getInetAddress());

        //Un hilo nuevo por cada cliente que se conecta.
        Thread hilo = new Thread(() -> {
            try {
                atenderCliente(cliente);
            } catch (Exception e) {
                System.out.println("Error inesperado: " + e.getMessage());
            }
        });

        hilo.start();
    }
}
```

Método servidor: atenderCliente -> Es el núcleo de cada conexión: pide el nombre, registra al cliente y procesa todos sus mensajes en un bucle while.

```
public static void atenderCliente(Socket cliente) throws Exception {

    DataInputStream entrada = new DataInputStream(cliente.getInputStream());
    DataOutputStream salida = new DataOutputStream(cliente.getOutputStream());

    //Pedir nombre.
    salida.writeUTF("Ingresá tu nombre de usuario: ");
    String deseado = entrada.readUTF().trim();
    String nombre = asignarNombre(deseado); //Nombre único garantizado.

    //Registrar al cliente en el mapa compartido.
    clientes.put(nombre, salida);

    //Avisar si el nombre tuvo que cambiar.
    String avisoNombre; //Creamos la variable de aviso.

    //Lo que se pregunta el sistema es: ¿El nombre final es exactamente igual al que el usuario deseaba?.
    if(nombre.equals(deseado)){
        //Si son iguales (verdadero), no hay que avisar nada.
        avisoNombre = "";
    } else{
        //Si son distintos (falso), armamos el mensaje de advertencia.
        avisoNombre = "\n " + deseado + " " ya existía. Tu nombre asignado es: " + nombre;
    }
}
```

Joaquín Devige
CLIENTE - SERVIDOR

```
//Menú de bienvenida.
salida.writeUTF(
    "\n===== \n"
    + " ¡BIENVENIDO AL PREGUNTADOS!\n"
    + "===== \n"
    + " Tu usuario: " + nombre + avisoNombre + "\n\n"
    + menu()
);

System.out.println("Registrado: " + nombre);
broadcast("-> " + nombre + " se conectó.", nombre); //Avisa a los demás.

//Variables del juego (cada hilo tiene las suyas, no se mezclan).
int puntaje = 0;
int idx = 0;
boolean juegoActivo = false; //El juego empieza solo si el cliente escribe jugar.

//Bucle principal.
while (true){

    String msg = entrada.readUTF().trim(); //Aca es donde el servidor atrapa la respuesta que el usuario envió.
    System.out.println("[ " + nombre + " ] " + msg); //Print en consola del servidor.

    //SALIR.
    if(msg.equalsIgnoreCase("SALIR")){
        salida.writeUTF("¡Hasta luego, " + nombre + "!");
        break;
    }

    //AYUDA.
    if(msg.equalsIgnoreCase("AYUDA")){
        salida.writeUTF(menu());
        continue;
    }
}
```

```
//FECHA.
if(msg.equalsIgnoreCase("FECHA")){

    String fecha = new SimpleDateFormat("dd/MM/yyyy HH:mm:ss").format(new Date());
    salida.writeUTF("Fecha y hora: " + fecha);
    continue;
}

//LISTA.
if(msg.equalsIgnoreCase("LISTA")){

    salida.writeUTF("Clientes conectados:\n" + listarClientes());
    continue;
}

//CALC <expresión>.
if(msg.toUpperCase().startsWith("CALCULAR ")){

    String expr = msg.substring(5).trim();
    salida.writeUTF(calcular(expr));
    continue;
}
```

```
/*ALL <mensaje>: broadcast a todos.

if(msg.toUpperCase().startsWith("*ALL ")){

    String texto = msg.substring(5).trim();

    if(texto.isEmpty()){

        salida.writeUTF(" El mensaje está vacío. Uso: *ALL hola a todos");
    } else{

        broadcast "[" + nombre + " -> TODOS]: " + texto, nombre);
        salida.writeUTF(" Mensaje enviado a todos.");
    }
    continue;
}
```

```
/*usuario <mensaje>: mensaje privado.

if(msg.startsWith("*")){

    //Formato esperado: *Juan hola como estás.
    String[] partes = msg.substring(1).split(" ", 2); // "Juan hola como estas", Saca el asterisco del principio.
    String destino = partes[0]; //Juan.
    String texto;

    //Si el mensaje que escribe el usuario es mayor a un, es decir
    //que hay dos partes lo que seria correcto lo toma, sino manda una cadena vacia para avisar del error.
    if(partes.length > 1){

        texto = partes[1].trim();
    } else{

        texto = "";
    }
}
```

Joaquín Devige

CLIENTE - SERVIDOR

```
//Si el texto queda vacio se avisa el formato correcto; Ejemplo: *Juan.
if(texto.isEmpty()){

    salida.writeUTF(" Formato: *usuario mensaje");

//Se valida que no se pueda escribir a uno mismo. Compara el destinatario con el nombre actual.
//El ignoreCase evita que *juan y *Juan pasen como destinatarios distintos.
} else if(destino.equalsIgnoreCase(nombre)){

    salida.writeUTF(" No podés mandarte mensajes a vos mismo.");
} else{
    //Busca al destinatario en el servidor con el concurrentHashMap.
    //Si no lo encuentra devuelve null.
    DataOutputStream salidaDestino = clientes.get(destino);

    if(salidaDestino != null){
        // El destinatario existe: enviar el mensaje normalmente.
        salidaDestino.writeUTF "[" + nombre + "]: " + texto);
        salida.writeUTF(" Mensaje enviado a " + destino + ".");
    } else{

        //El destinatario NO existe: se busca una alternativa.
        //Busca al primer usuario conectado que no sea el emisor.
        //keySet: Extrae solo las claves del mapa (HashMap), es decir, solo los nombre.
        //Devuelve un conjunto.
        //stream: Convierte ese conjunto en una fila de elementos para procesar uno por uno.
        //Se usa un filtro: recorre cada elemento del stream y descarta si no cumplen la condicion.
        //n: Cada nombre (Juan, Ana, Pedro).
        //!n.equals(nombre): Pregunta si este nombre actual si es el emisor, sino lo descarta y continua.
        //Devuelve el primero que encuentra.
        //orElse: si tenia un valor lo devuelve, si estaba vacio devuelve null.
        String alternativa = clientes.keySet().stream().filter(n -> !n.equals(nombre)).findFirst().orElse(null);
```

```
        if(alternativa != null){
            clientes.get(alternativa).writeUTF( "[" + nombre + " (redirigido de '" + destino + "'" + "): " + texto);
            salida.writeUTF(" '" + destino + "' no existe. Mensaje redirigido a '" + alternativa + "'");
        } else{

            salida.writeUTF(destino + "' no existe y no hay otros usuarios conectados.");
        }
    }
}
continue;
}

//JUGAR: inicia el Preguntados.

if(msg.equalsIgnoreCase("JUGAR")){

    juegoActivo = true;
    puntaje = 0;
    idx = 0;
    salida.writeUTF("¡BIENVENIDO AL PREGUNTADOS!\nResponde con A, B, C o D.\n\n" + formatear(idx, puntaje));
    continue;
}
```

Joaquín Devige

CLIENTE - SERVIDOR

```
//Respuesta del juego (A / B / C / D).
if(juegoActivo && msg.toUpperCase().matches("[ABCD]")){
    boolean correcto = msg.toUpperCase().equals(PREGUNTAS[idx][5]);
    String feedback;

    if(correcto){
        puntaje++;
        feedback = "¡CORRECTO! Puntaje: " + puntaje + "/" + PREGUNTAS.length + "\n\n";
    } else{
        feedback = "INCORRECTO. Era: " + PREGUNTAS[idx][5] + " | Puntaje: " + puntaje + "/" + PREGUNTAS.length + "\n\n";
    }

    idx++;

    if(idx < PREGUNTAS.length){
        salida.writeUTF(feedback + formatear(idx, puntaje));
    } else{
        salida.writeUTF(feedback + "=== FIN DEL JUEGO ===\nPuntaje final: " + puntaje + "/" + PREGUNTAS.length + "\nEscribi AYUDA para ver comandos.");
        juegoActivo = false;
    }
    continue;
}

//Cualquier otra cosa.
salida.writeUTF(" Comando no reconocido. Escribi AYUDA para ver los comandos.");
}

//Limpieza al desconectar.
clientes.remove(nombre);
cliente.close(); //Cierra las conexiones entre el servidor y ese cliente en particular.
System.out.println("Desconectado: " + nombre);
broadcast("-> " + nombre + " se desconectó.", nombre); //Manda este mensaje a todos los usuarios conectados.
} //Fin bucle while.
```

Método del servidor: **asignarNombre** -> Garantiza que cada usuario tenga un nombre único; si el nombre ya existe, le agrega un número (ej: "Juan2").


```
//Funcionamiento: Le asigna un sufijo a los nombres ya existentes y repetidos.
public static synchronized String asignarNombre(String deseado){

    String nombreBase;

    //Se valida si el texto ingresado está vacío.
    if (deseado.isEmpty()){

        nombreBase = "Usuario"; //Le damos un nombre por defecto.
    } else{

        nombreBase = deseado;    //Usamos el nombre que eligió.
    }

    //Se verifica si ese nombre está libre en el servidor.
    boolean elNombreEstaOcupado = clientes.containsKey(nombreBase);

    if (elNombreEstaOcupado == false){

        //Si no está ocupado, devolvemos el nombre tal cual está y terminamos aquí.
        return nombreBase;
    }
}
```

```
//Si se llegó a este punto, significa que el nombre ya existe.
//Empezamos a buscar una alternativa agregando un número al final.
int numeroSufijo = 2;
String nombreAlternativo = nombreBase + " " + numeroSufijo; // Ejemplo: "Juan2"

//Bucle: Mientras el nombre alternativo siga existiendo en la lista.
while(clientes.containsKey(nombreAlternativo)){

    // Incrementamos el número (pasa de 2 a 3, luego a 4, etc.)
    numeroSufijo = numeroSufijo + 1;

    // Volvemos a armar el nombre para que el bucle lo vuelva a evaluar
    nombreAlternativo = nombreBase + numeroSufijo;
}

//Cuando el bucle termina, es porque se encontro un nombre libre.
return nombreAlternativo;
}
```

Método del servidor: **broadcast** -> Manda un mensaje a todos los clientes conectados, excepto al que lo envió.

Joaquín Devige CLIENTE - SERVIDOR

```
//Manda un mensaje a todos excepto al emisor.
public static void broadcast(String msg, String emisor){

    //Se recorre el mapa de clientes uno por uno.
    //Map.Entry representa un "par" de datos dentro de nuestro mapa (NombreUsuario: FlujoDeSalida).
    for(Map.Entry<String, DataOutputStream> parCliente : clientes.entrySet()){

        //Extraemos los datos del cliente actual del bucle en variables separadas
        String nombreDestinatario = parCliente.getKey();           //La llave (el nombre).
        DataOutputStream salidaDestinatario = parCliente.getValue(); //El valor (para enviar los datos).

        //Comparamos el nombre del cliente actual con el nombre del emisor.
        boolean esElMismoEmisor = nombreDestinatario.equals(emisor);

        //Si no es la persona que mandó el mensaje original, se lo enviamos.
        if(esElMismoEmisor == false){

            //Intentamos enviar el mensaje.
            //Usamos try-catch porque las conexiones de red son inestables.
            try {
                salidaDestinatario.writeUTF(msg);
            } catch (Exception errorConexion) {
                //Si falla (ej. al destinatario se le cortó el internet justo en ese milisegundo),
                //el catch está vacío de forma intencional.
                //Esto permite ignorar el error de este usuario en particular y
                //que el bucle 'for' continúe enviando el mensaje al resto de la lista sin que el servidor se caiga.
            }
        }
    }
}
```

Método del servidor: listarClientes -> Devuelve la lista de nombres de todos los usuarios conectados en ese momento.

```
//Lista los clientes conectados.
public static String listarClientes(){

    //Primero verificamos si el mapa está vacío (si no hay nadie conectado).
    boolean noHayNadie = clientes.isEmpty();

    if(noHayNadie == true){
        //Si es verdad, devolvemos un texto avisando y terminamos el método aquí.
        return " (ninguno conectado)";
    }

    //Si llegamos acá, significa que SÍ hay clientes.
    //Usamos StringBuilder porque es la forma más eficiente y rápida de
    //unir textos en Java cuando estamos dentro de un bucle.
    StringBuilder listaArmada = new StringBuilder();

    //Extraemos SOLO los nombres del mapa.
    //keySet() nos da un conjunto (Set) con todas las llaves (los nombres de usuario).
    Set<String> nombresConectados = clientes.keySet();

    //Recorremos ese conjunto de nombres uno por uno.
    for(String nombreDelCliente : nombresConectados){

        //Vamos agregando pedazos de texto al StringBuilder:
        listaArmada.append(" -- ");
        listaArmada.append(nombreDelCliente); //Agregamos el nombre del cliente.
        listaArmada.append("\n"); //Agregamos un salto de línea para el siguiente.
    }

    //Se convierte el objeto StringBuilder en un texto normal (String).
    String textoFinal = listaArmada.toString();

    //El método trim() borra los espacios o saltos de línea que sobran al principio o al final.
    //(Esto evita que quede un salto de línea vacío al final de la lista).
    return textoFinal.trim();
}
```

Método del servidor: **calcular** -> Recibe una expresión matemática como texto y devuelve el resultado.

Método del servidor: **log** -> Imprime un mensaje en la consola del servidor con la hora exacta.

Método del servidor: **menú** -> Devuelve el texto con todos los comandos disponibles para mostrarle al usuario.

```
//Resuelve una expresión matemática simple.
public static String calcular(String expr){

    try {
        //Evalúa expresiones básicas: +, -, *, /, paréntesis.
        javax.script.ScriptEngine js = new javax.script.ScriptEngineManager().getEngineByName("JavaScript");
        Object resultado = js.eval(expr);
        return " " + expr + " = " + resultado;
    } catch (Exception e) {
        return "Expresión inválida: " + expr;
    }
}

//Log con hora en consola del servidor.
public static void log(String msg){

    String hora = new SimpleDateFormat("HH:mm:ss").format(new Date());
    System.out.println "[" + hora + " ] " + msg;
}

//Menú de ayuda.
public static String menu(){

    return " COMANDOS:\n"
        + "   JUGAR           -> Iniciar el juego Preguntados\n"
        + "   FECHA           -> Ver fecha y hora actual\n"
        + "   LISTA           -> Ver clientes conectados\n"
        + "   CALCULAR        -> Resolver expresión matemática\n"
        + "   *NombreUsuario msg -> Mensaje privado\n"
        + "   *ALL msg         -> Mensaje a todos\n"
        + "   AYUDA           -> Ver este menú\n"
        + "   SALIR           -> Desconectarse\n"
        + "   _____";
}
}
```

Método del servidor: formatear -> Arma el texto de cada pregunta del juego con sus opciones y el puntaje actual.

```
//Formatea la pregunta del juego.
public static String formatear(int idx, int puntaje){

    String[] p = PREGUNTAS[idx];
    return " Pregunta " + (idx + 1) + " / " + PREGUNTAS.length + " | Puntaje: " + puntaje + " —\n" + p[0] + "\n" + p[1] + " " + p[2] + " " + p[3] + " " + p[4] +
        "\nTu respuesta: ";
}
}
```

CONSOLA

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
CALCULAR -> Resolver expresion matematica
*NombreUsuario msg -> Mensaje privado
*ALL msg -> Mensaje a todos
AYUDA -> Ver este menú
SALIR -> Desconectarse

fecha

[SERVIDOR] Fecha y hora: 23/04/2026 01:08:54
*all ¿como andan todos?

[SERVIDOR] Mensaje enviado a todos.

[SERVIDOR] -> joaquin se desconectó.
jugar

[SERVIDOR] ¡BIENVENIDO AL PREGUNTADOS!
Responde con A, B, C o D.

Pregunta 1/10 | Puntaje: 0 —
¿Cuánto es 5 + 3?
A)6 B)8 C)9 D)7
Tu respuesta:
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
[SERVIDOR] -> sebastian se conectó.
[SERVIDOR] -> miguel se conectó.
calcular 4*6

[SERVIDOR] 4*6 = 24

[SERVIDOR] [miguel -> TODOS]: ¿como andan todos?
ayuda

[SERVIDOR] COMANDOS:
JUGAR -> Iniciar el juego Preguntados
FECHA -> Ver fecha y hora actual
LISTA -> Ver clientes conectados
CALCULAR -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg -> Mensaje a todos
AYUDA -> Ver este menú
SALIR -> Desconectarse

[SERVIDOR] -> joaquin se desconectó.
```

Joaquín Devige
CLIENTE - SERVIDOR

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
run:
Servidor iniciado en puerto 5500...
Nueva conexión desde:/127.0.0.1
Registrado: joaquin
[joaquin] calcular 4*3
Nueva conexión desde:/127.0.0.1
Nueva conexión desde:/127.0.0.1
Nueva conexión desde:/127.0.0.1
Registrado: jose
Registrado: sebastian
Registrado: miguel
[miguel] fecha
[sebastian] lista
[jose] calcular 4*6
[joaquin] *sebastian como estas?
[miguel] *all ¿como andan todos?
[jose] ayuda
[joaquin] salir
Desconectado: joaquin
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
[SERVIDOR] -> jose se conectó.

[SERVIDOR] -> sebastian se conectó.
*sebastian como estas?

[SERVIDOR] Mensaje enviado a sebastian.

[SERVIDOR] [miguel -> TODOS]: ¿como andan todos?
salir
Conexión cerrada.

Conexión cerrada por el servidor.
BUILD SUCCESSFUL (total time: 3 minutes 13 seconds)
```

Joaquín Devige
CLIENTE - SERVIDOR

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
CALCULAR -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg -> Mensaje a todos
AYUDA -> Ver este menú
SALIR -> Desconectarse

[SERVIDOR] -> sebastian se conectó.

[SERVIDOR] -> miguel se conectó.
calcular 4*6

[SERVIDOR] 4*6 = 24

[SERVIDOR] [miguel -> TODOS]: ¿como andan todos?
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
[SERVIDOR] 4*6 = 24

[SERVIDOR] [miguel -> TODOS]: ¿como andan todos?
ayuda

[SERVIDOR] COMANDOS:
JUGAR -> Iniciar el juego Preguntados
FECHA -> Ver fecha y hora actual
LISTA -> Ver clientes conectados
CALCULAR -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg -> Mensaje a todos
AYUDA -> Ver este menú
SALIR -> Desconectarse
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
FECHA -> Ver fecha y hora actual
LISTA -> Ver clientes conectados
CALCULAR -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg -> Mensaje a todos
AYUDA -> Ver este menú
SALIR -> Desconectarse

fecha

[SERVIDOR] Fecha y hora: 23/04/2026 01:08:54
*all ¿como andan todos?

[SERVIDOR] Mensaje enviado a todos.
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
[SERVIDOR] -> miguel se conectó.
lista

[SERVIDOR] Clientes conectados:
-- jose
-- miguel
-- sebastian
-- joaquin

[SERVIDOR] [joaquin]: como estas?

[SERVIDOR] [miguel -> TODOS]: ¿como andan todos?
```



```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
AYUDA -> Ver este menú
SALIR -> Desconectarse
=====
calcular 4*3

[SERVIDOR] 4*3 = 12

[SERVIDOR] -> jose se conectó.

[SERVIDOR] -> sebastian se conectó.

[SERVIDOR] -> miguel se conectó.
*sebastian como estas?

[SERVIDOR] Mensaje enviado a sebastian.
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
LISTA -> Ver clientes conectados
CALCULAR -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg -> Mensaje a todos
AYUDA -> Ver este menú
SALIR -> Desconectarse
=====

[SERVIDOR] -> sebastian se conectó.

[SERVIDOR] -> miguel se conectó.
calcular 4*6

[SERVIDOR] 4*6 = 24
```

Joaquín Devige
CLIENTE - SERVIDOR

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
//Evaluá expresiones básicas: +, -, *, /, parentesis.

COMANDOS:
JUGAR          -> Iniciar el juego Preguntados
FECHA          -> Ver fecha y hora actual
LISTA          -> Ver clientes conectados
CALCULAR       -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg       -> Mensaje a todos
AYUDA          -> Ver este menú
SALIR          -> Desconectarse

=====

fecha

[SERVIDOR] Fecha y hora: 23/04/2026 01:08:54
```

```
Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x HilosYSockets (run) #5 x
//Evaluá expresiones básicas: +, -, *, /, parentesis.

CALCULAR       -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg       -> Mensaje a todos
AYUDA          -> Ver este menú
SALIR          -> Desconectarse

=====

[SERVIDOR] -> miguel se conectó.
lista

[SERVIDOR] Clientes conectados:
-- jose
-- miguel
-- sebastian
-- joaquin
```

Joaquín Devige
CLIENTE - SERVIDOR

```
{ Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x
[SERVIDOR]
¡BIENVENIDO AL PREGUNTADOS!
Tu usuario: miguel
COMANDOS:
JUGAR          -> Iniciar el juego Preguntados
FECHA          -> Ver fecha y hora actual
LISTA          -> Ver clientes conectados
CALCULAR       -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg       -> Mensaje a todos
AYUDA          -> Ver este menú
SALIR          -> Desconectarse
fecha
[SERVIDOR] Fecha y hora: 23/04/2026 00:59:44
lista
[SERVIDOR] Clientes conectados:
-- santiago
-- miguel
-- joaquin
```

```
{ Output
HilosYSockets (run) x HilosYSockets (run) #2 x HilosYSockets (run) #3 x HilosYSockets (run) #4 x
run:
¡Conectado al servidor!
[SERVIDOR] Ingresá tu nombre de usuario:
miguel
[SERVIDOR]
¡BIENVENIDO AL PREGUNTADOS!
Tu usuario: miguel
COMANDOS:
JUGAR          -> Iniciar el juego Preguntados
FECHA          -> Ver fecha y hora actual
LISTA          -> Ver clientes conectados
CALCULAR       -> Resolver expresión matemática
*NombreUsuario msg -> Mensaje privado
*ALL msg       -> Mensaje a todos
AYUDA          -> Ver este menú
SALIR          -> Desconectarse
fecha
[SERVIDOR] Fecha y hora: 23/04/2026 00:59:44
```